

SLIDE 1 — What is a Character Array?

Definition:

A **character array** is an array that stores multiple **characters**. It is commonly used to store **strings** in C.

Simple idea

Characters → **Character Array** → **String**

Example:

```
char name[6] = {'V', 'i', 'c', 'k', 'y', '\0'};
```

Explanation:

Index:	0	1	2	3	4	5
	[V]	[i]	[c]	[k]	[y]	[\0]

\0 marks the **end of the string**.

 **Remember: String = Characters + \0**

SLIDE 2 — What is a String?

Definition:

A **string** is a sequence of characters stored in a character array and ended by a **null character** \0.

Example

```
char name[] = "Vicky";
```

C automatically stores:

```
V i c k y \0
```

Explanation:

The string "Vicky" has **5 visible characters**, but the array needs **6 positions** including `\0`.

★ Remember

- `char` → Stores one character
 - `char array` → Stores multiple characters
 - `\0` → Marks the end of a string
-

SLIDE 3 — Accessing Characters

Each character can be accessed using its **index**.

Example

```
char name[] = "Vicky";
```

```
printf("%c", name[0]);  
printf("%c", name[2]);
```

Explanation:

```
Index: 0  1  2  3  4  
      [V] [i] [c] [k] [y]  
      ↑   ↑  
      name[0] name[2]
```

Output:

Vc

🧠 Remember: `name[index]` → Access one character

SLIDE 4 — Reading & Displaying Strings

Displaying a string

```
char name[] = "Vicky";
```

```
printf("%s", name);
```

Output:

Vicky

`%s` is used to print a **string**.

Reading a string

```
char name[20];
```

```
scanf("%s", name);
```

If the input is:

Vicky

The array stores:

V i c k y \0

Basic Pattern

DECLARE → STORE → ACCESS → DISPLAY

SLIDE 5 — RECAP

Character Array = String! 

Flow

CHARACTERS



CHARACTER ARRAY



ADD \0



STRING



ACCESS / DISPLAY

5 Things to Remember

1. Character arrays store characters
2. Strings are stored using character arrays
3. Strings end with `\0`
4. Indexing starts from 0
5. `%s` is used to display a string

Final Example

```
char name[] = "Hello";  
  
printf("%c", name[1]);
```

Index 1 → `e`

Output:

e

Memory Trick

`char` → ONE character 

`char[]` → MANY characters 

`\0` → STRING END 

QUESTIONS

1. What is a character array?

- A) Array of integers
- B) Array of characters
- C) Array of floats
- D) Array of conditions

 **Hint:** It stores characters.

 **Glossary: Character** → A single symbol such as **A**, **b**, or **5**.

 **Answer: B**

2. What marks the end of a string in C?

- A) `\n`
- B) `\t`
- C) `\0`
- D) `\b`

 **Hint:** C uses a special null character to mark the string's end.

 **Glossary: Null character** → `\0`, which marks the end of a C string.

 **Answer: C) `\0`**

3. What is the output?

```
char name[] = "Hello";
```

```
printf("%c", name[1]);
```

 **Hint:** Indexing starts from 0.

 **Glossary: String** → A sequence of characters ending with `\0`.

 **Answer: e**

4. Fill in the blank

Complete the code to print the entire string:

```
char name[] = "Vicky";
```

```
printf("_____", name);
```

 **Hint:** Use the format specifier for strings.

 **Glossary: Format specifier** → A symbol such as `%c` or `%s` that tells `printf()` what type of data to display.

 **Answer: `%s`**

5. How many positions are needed here?

```
char name[] = "Hello";
```

- A) 4
- B) 5
- C) 6
- D) 7

 **Hint:** Count the 5 characters plus `\0`.

 **Glossary: Position** → A location identified by an index.

 **Answer: C) 6**

6. Which is used to access one character from a string?

- A) `name[index]`
- B) `name()`
- C) `name{index}`
- D) `name<index>`

 **Hint:** Arrays use square brackets.

 **Glossary: Access** → Retrieve a particular value.

 **Answer: A) `name[index]`**

7. Arrange the steps in the correct order:

- A. Store characters
- B. Create character array
- C. End with `\0`
- D. Use as a string

 **Hint:** Create → Store → End → Use.

 **Glossary:** String → A sequence of characters stored in a character array.

 **Answer:** B → A → C → D